The Ultimate Loon-E Guidebook

Humber Polytechnic

Version 1.0.0
Date: May 12, 2026

Contents

List of Figures

List of Tables

1 Introduction

Humber ASV is a multidisciplinary team of passionate Humber Polytechnic students and professionals dedicated to advancing autonomous maritime technology.[2] The team is focused on designing, building, and operating their autonomous surface vehicle (ASV) to contribute to the field of maritime robotics. The Loon-E ASV is one of the key projects undertaken by HumberASV, showcasing the team's commitment to innovation and excellence in autonomous maritime technology.[2] This document serves as a comprehensive guidebook for the Loon-E ASV, providing detailed information on its design, operation, and maintenance. It is intended to be a valuable resource for stakeholders and developers involved in the Loon-E project, offering insights into the ASV's capabilities, specifications, and best practices for its use and upkeep. The on-board computer system includes hardware components such as the Jetson Orin computer, Zedx camera, and various student-lead designs [7].

Document Conventions

Throughout this document, we will use the following conventions to highlight information:

Infobox Used to provide additional information or context about a topic.

Warning Box Used to highlight important warnings or cautions that users should be aware of.

Note Box Used to provide additional notes or clarifications about a topic.

Footnotes Used to provide additional information or references related to specific points in the text.

Hyperlinks Used to direct readers to external resources or related documents for further information. Hyperlinks are displayed in a distinct blue color to differentiate them from the base text.

INFO

This is an infobox. It is used to provide additional information or context about a topic.

WARNING

This is a warning box. It is used to highlight important warnings or cautions that users should be aware of.

NOTE

This is a note box. It is used to provide additional notes or clarifications about a topic.

Use of This Document

This document is intended to be a single source of truth for all information related to the Loon-E ASV. It is designed to be comprehensive and detailed, providing all necessary information for stakeholders and developers involved in the project. Readers are encouraged to read though sections of interest instead of the entire document, as it is structured in a way that allows for easy navigation to specific topics. The document is also intended to be a living document, meaning that it will be updated and revised as new information becomes available or as the project evolves. Users are encouraged to refer back to this document regularly to stay informed about the latest developments and updates related to the Loon-E ASV.

Development of This Document

This document is developed using LaTeX, a typesetting system that allows for the creation of professional and well-structured documents. While there is a technical learning curve associated with using LaTeX, it offers significant benefits in terms of document organization, formatting, and the ability to easily include complex elements such as figures, tables, and references. Most of the content is written in Latex files and then compiled into a PDF format for distribution. The source files are organized in a modular fashion for easy editing and updating as needed.

Legal

This document is a documentation file licensed under the MIT License, which allows for free use, modification, and distribution of the content, provided that the original copyright notice and permission notice are included in all copies or substantial portions of the Software.

> **INFO**
>
> The documentation is associated with the Software under MIT License, which means that the documentation itself is licensed under MIT and may contain references to the software that is also licensed under MIT. Users should ensure that they comply with the terms of the MIT License when using or distributing the software and its associated documentation.

1.4.1 MIT License

2 Systems Overview

The Loon-E ASV is an autonomous surface vehicle designed to operate on water. It is not equipped to be underwater; please don't try to submerge it. Loon-E is designed to be modular and adaptable, allowing for various configurations and payloads to suit different tasks as competition requirements evolve [7].

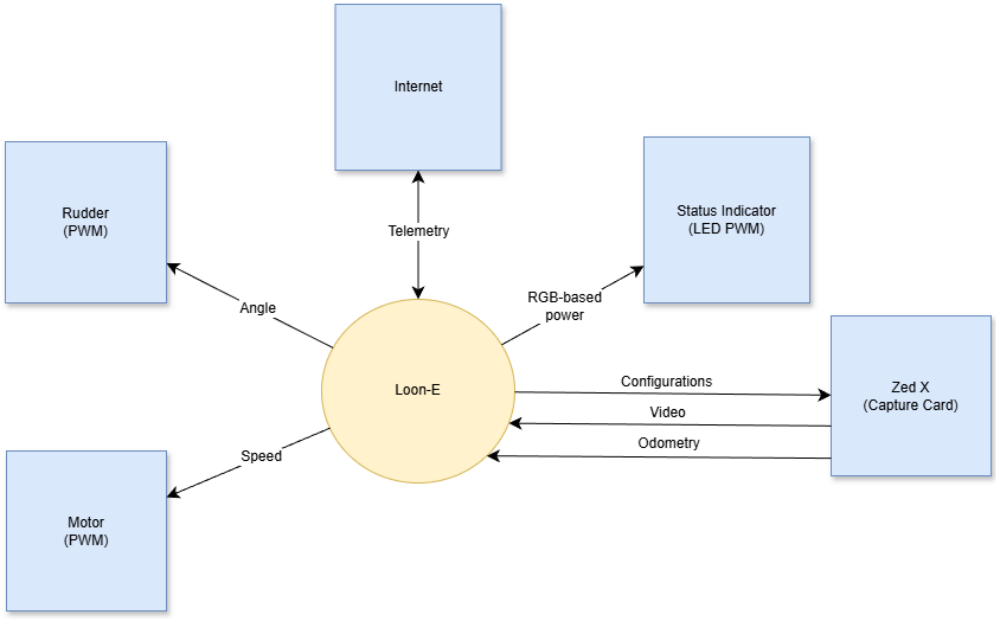

Figure 1: System Context Diagram

3 Software System

The software system of the Loon-E ASV is responsible for controlling the vehicle's operations, including navigation, sensor data processing, and communication. The entire system can be illustrated in Figure 2, which illustrates a 3 process system consisting of a base station, the primary computer, and a web client. These 3 components work together to realize the software system of the ASV.

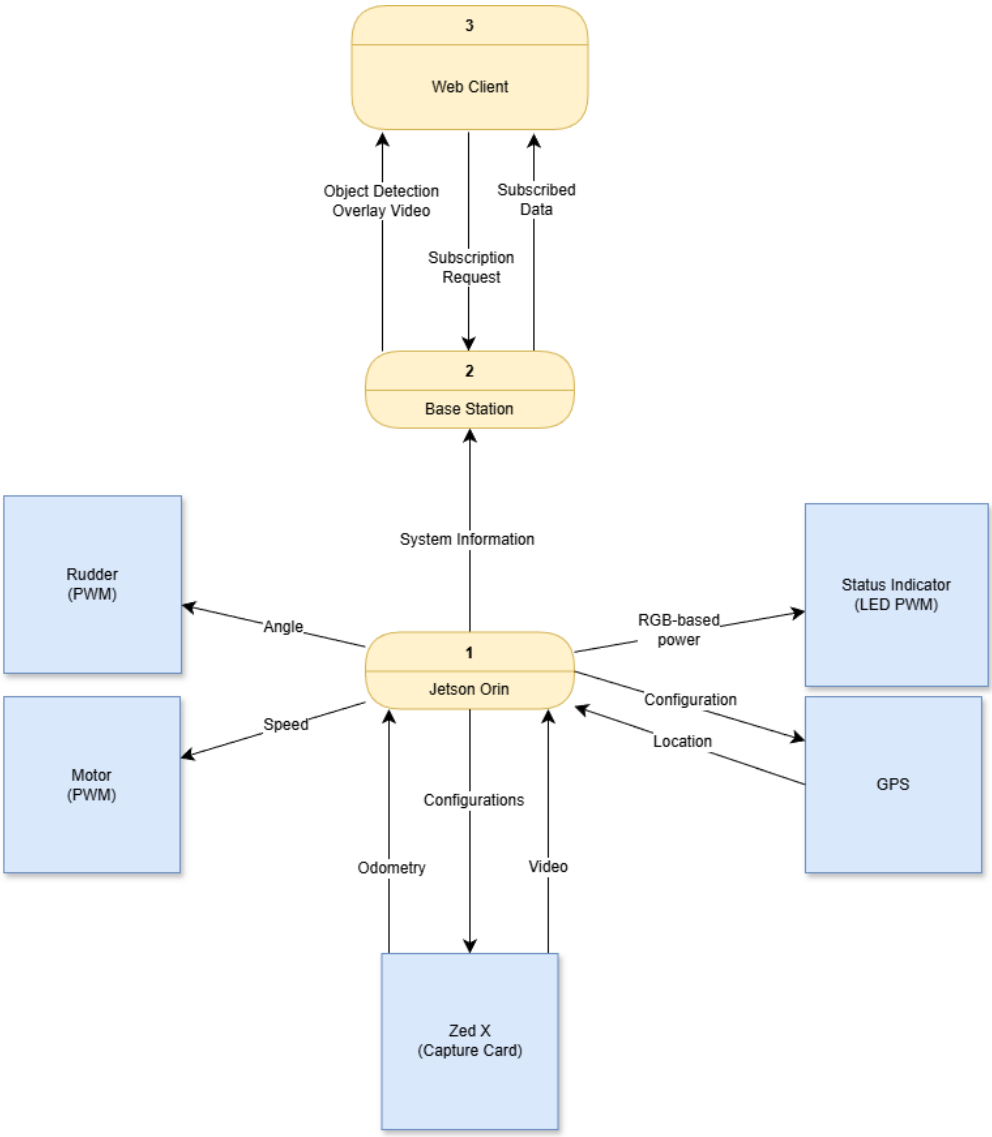

Figure 2: Level 1 Data Flow Diagram

The Loon-E ASV system sends a variety of data which need to be remotely accessed and visualized by users. This data must be transmitted from the ASV to the Base Station, which will then be accessed by users through a web client application.

Primary Computer

The primary computer is responsible for controlling the ASV's operations, including navigation, sensor data processing, and communication.

Technology	Purpose
ROS2 Humble	Robotics middleware for communication and control
Ubuntu Jazzy Jellyfish	Operating system for the primary computer
Cyclone DDS	(ROS middleware) Chosen data distribution service[1]
Zedx SDK	Software development kit for the Zedx camera
Docker	Containerization platform for software deployment

Table 1: Primary Computer Software Stack

3.1.1 ROS2 Humble

ROS2 Humble is a robotics middleware that provides a framework for building robot applications. It offers a collection of tools, libraries, and conventions that simplify the development of complex robotic systems [6]. We designed a series of nodes–processes that perform specific tasks–to run on the primary computer, which communicate with each other using ROS2 topics and services. Figure 3 illustrates a level 1.1 data flow diagram for the primary computer, which breaks down the primary computer process from Figure 2 into its constituent nodes and their interactions.

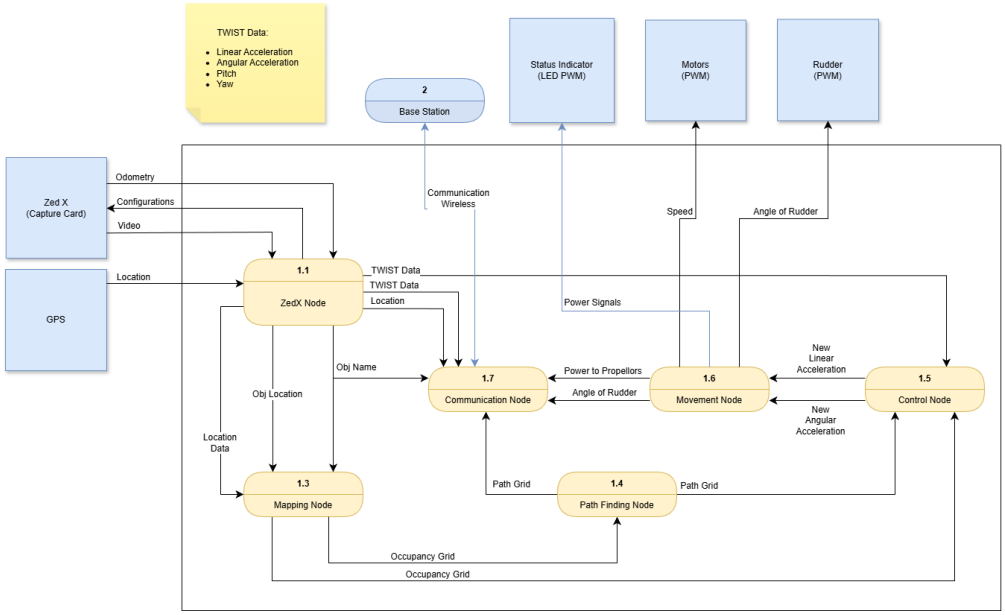

Figure 3: Level 1.1 Data Flow Diagram for Primary Computer

To test and send data from the primary computer to the base station, we set up a ROS2 publisher node that sends simulated sensor data to a ROS2 subscriber node running on the base station. Additionally, for testing purposes, we set up a ROS2 subscriber node on a secondary server computer that runs a simulation on Isaac SIM.

[1]ROS2 supports multiple DDS implementations, however, Stereolabs, the manufacturer of the Zedx camera, only provides support for Cyclone DDS [9] [4].

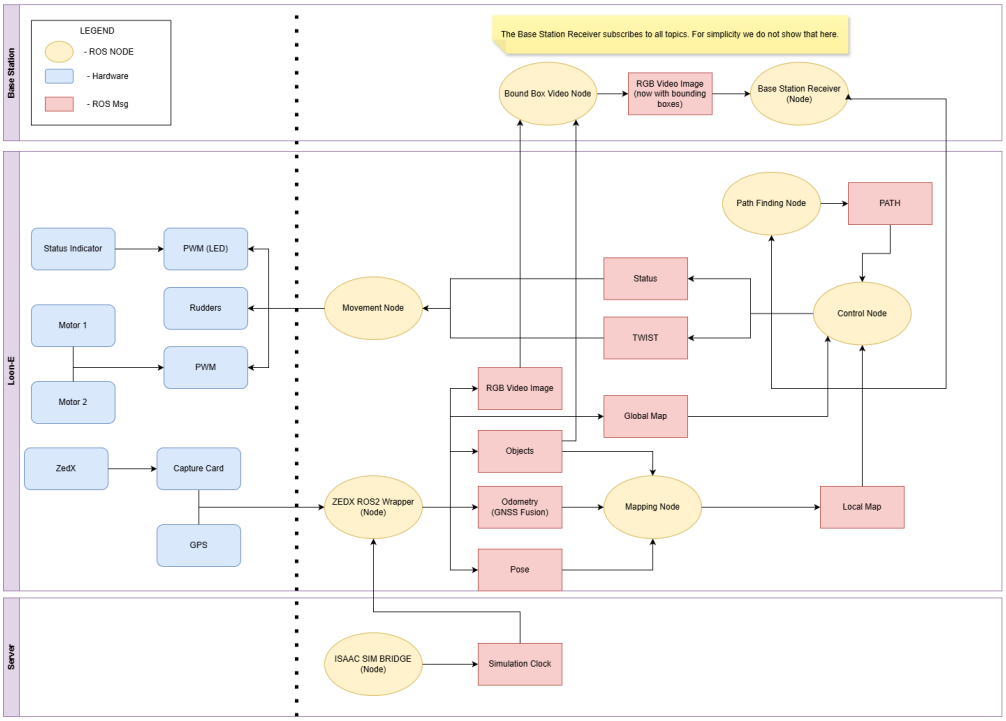

Figure 4: Software Architecture Diagram

The web client provides a user interface for monitoring the ASV remotely. The base station is responsible for receiving data from the ASV for diagnostic monitoring and hosting the web client.

4 Base Station

The Base Station is a critical component of the Loon-E ASV's software system, responsible for receiving data from the ASV and providing it to users on a web client application. It acts as an intermediary between the ASV and the users, allowing for remote access to the ASV's data and operations.
The system will implement the connected access point (TP-Link Omada AC1200) to become a wireless access point allowing the ASV to connect to the base station. The TP-Link Omada access point provides the wireless link between the ASV and the local network. The access point connects by Ethernet to a switch, which uplinks to a router or Internet gateway. The base station computer stays on the same network and relays ASV data to the web client, but it does not control the ASV or perform navigation. In other words, The base station acts as a server that receives ASV data and serves it to the web client, while the access point, switch, and router provide the network connectivity. It is designed to meet the specifications required for participation in maritime robotics competitions such as RoboBoat and Njord

Base Station Software Stack

The Base Station can be a Windows, Linux, or macOS computer running ROS2 Humble, which is the same version of ROS2 used on the ASV's computer.

> **⚠ WARNING**
>
> The base station must be on the same network, DDS, Domain ID, and ROS2 version as the ASV's computer to receive data from the ASV.

Using CycloneDDS as the data distribution service, the base station will subscribe to all topics published by the ASV, allowing it to receive and process all relevant data from the ASV's operations. The base station will then relay this data to the web client application for visualization and monitoring by users.

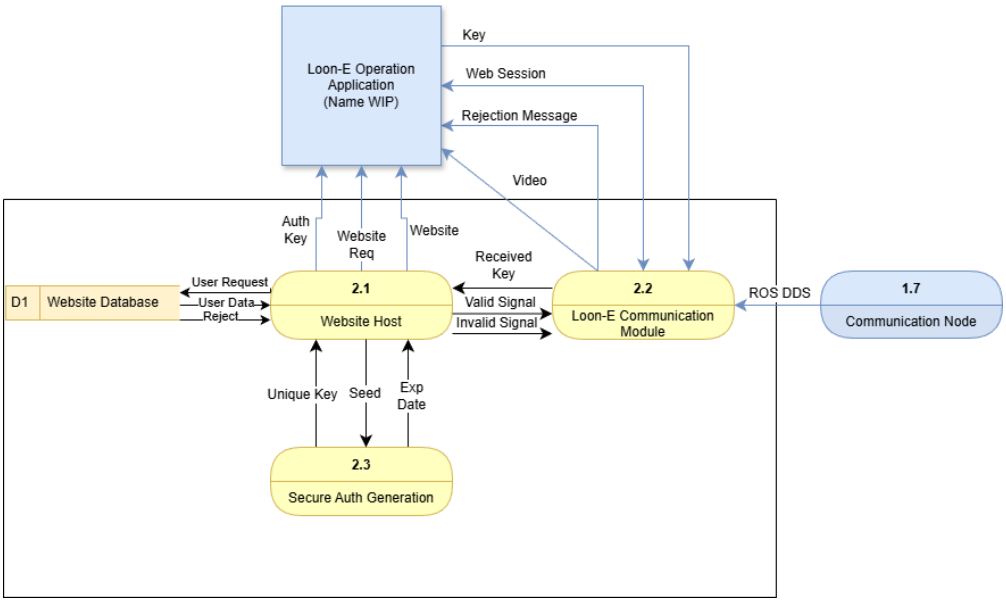

Figure 5: Base Station System Diagram

> **INFO**
>
> The base station is not responsible for controlling the ASV or computing navigation paths; it only receives and displays data from the ASV.

4.1.1 Previous Implementation

Previous implementation of the basestation used MQTT and Django running through containers to receive data from the ASV and serve it to the web client application. However, this approach was found to be less efficient and more complex than using ROS2's built-in communication capabilities with CycloneDDS. By using ROS2 and CycloneDDS, we can simplify the architecture of the base station and improve the performance of data transmission between the Base Station and the ASV. Additionally, we had difficulty with RTP streaming of the Zedx camera feed due to problems with the Zedxros2Wrapper's streaming capabilities.[2] New addition will build on these lessons through the following changes to the architecture:

ROS2 with CycloneDDS Using ROS2's built-in communication capabilities with CycloneDDS allows for more efficient and streamlined data transmission between the ASV and the base station, eliminating the need for additional middleware like MQTT.

Web Client Application The web client will be simplified to Flask to reduce technical complexity.

MQTT Removal MQTT will be removed from the architecture to simplify the communication between the ASV and the base station, as ROS2 with CycloneDDS can handle all necessary data transmission without the need for additional middleware.

RTP Streaming Removal The RTP streaming of the Zedx camera feed will be removed due to the current limitations of the Zedxros2Wrapper's streaming capabilities, and instead, we will explore alternative methods for transmitting camera data from the ASV to the base station.

Web Client Application

The web client application will display telemetry data in a user-friendly interface–following figure 6–allowing users to monitor the ASV's status and performance during operations

[2] As of the time of writing, the Zedxros2Wrapper's streaming capabilities are not fully functional, see this issue for more details.

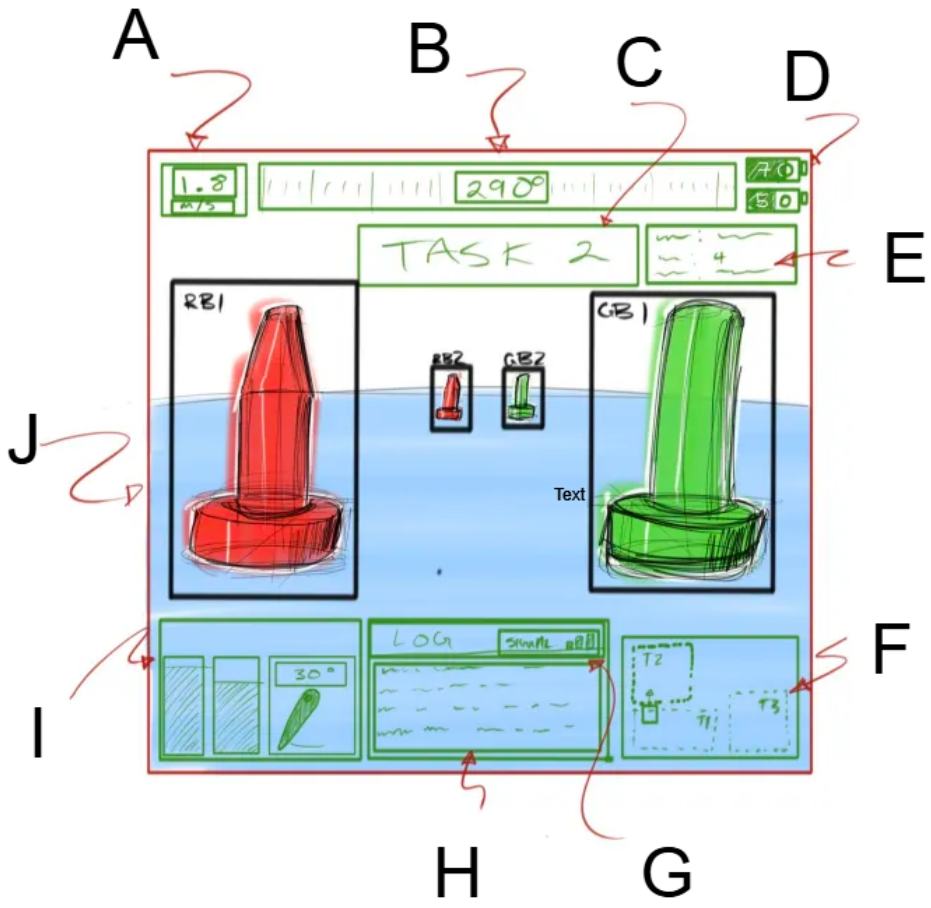

Figure 6: Web Client Application GUI Mockup

In the Figure 6 (above), alphabetical labels correspond to the following data displayed on the web client application:

A Speedometer
A speed over ground measurement in meters per second.

B COG heading
Course over ground (COG) with trail vs plot of ideal route given by GNSS points to compare.

C Task
The task name.

D Batteries: Motor and System in percentage
Power bars with percent detailing the power in two battery systems in Loon-E.

E Task Data
Contains data relevant to the current task such as a status indication, Latitude, Longitude.

F Map
Shows plot of ideal route given by GNSS points, task locations and the boat.

G Signal Strength

H Log
Log from the Loon-E.

I Power and Rudder Panel
Sideby side power bars detailing the power allocation in the propoller.

J Recieved Image Recognition
Object Detection windows from the Loon-E.[3]

The web client application will be accessible through a web browser and will not require any additional software installation for user access; however, the use of React for dynamic content and interactive features may require users to have a modern web browser that supports React applications.

> **INFO**
>
> Web Client users will also require the need of an internet connection to access any third party resources used in the web application, such as fonts, icons, or external APIs.

[3]To reduce processing load the ASV will only send the video feed, and locations of detected objects, the Base Station will be responsible for combining the two sources into one video feed and sending the results to the web client application for display.

5 Base Station Specifications

The basestation is responsible for receiving data from the ASV for diagnostic monitoring and hosting the web client.
It will host the following endpoints:

/admin The admin webpage that allows technical users to manage secure tokens for user access, including creating, revoking, and expiring tokens as needed to maintain security.

/admin/create_token An endpoint for the admin webpage to create secure tokens for user access.

/admin/revoke_token An endpoint for the admin webpage to revoke secure tokens for user access.

/admin/expire_token An endpoint for the admin webpage to expire secure tokens for user access.

/admin/tokens An endpoint for the admin webpage to get a list of all active tokens for user access.

/client The web client application to access the streaming data with a valid token.

/uh_oh An endpoint to handle unauthorized access attempts and display an appropriate error message to users who do not have valid tokens or permissions to access the application.

Django Implementation may still prove useful, but discussion over use of Django or Flask still exists. The use of Docker is garanteed anyways.

User Classes and Characteristics

The primary users of the system are members of the following:

Humber ASV Team The Humber ASV team is responsible for developing, testing, and operating the Loon-E ASV. They will use the base station and web client to monitor the ASV's status and performance during development and testing, as well as during competitions. The team will also use the base station to receive data from the ASV for diagnostic purposes and to analyze the ASV's performance.

Competition Judges Judges at maritime robotics competitions such as RoboBoat and Njord will use the web client application to monitor the ASV's status and performance during competitions. They will evaluate the ASV's performance based on the data displayed on the web client, such as speed, heading, task completion, and other relevant metrics.

Competing Teams Other teams participating in maritime robotics competitions may also use the web client application to monitor their own ASVs if they are using a similar architecture, or to compare their performance with that of Loon-E. They may also use it to observe Loon-E's performance during competitions.

Sponsors and Stakeholders Sponsors and stakeholders of the Humber ASV team may also be interested in monitoring the ASV's performance during testing and competitions. They may use the web client application to view telemetry data and assess the progress of the project.

User Needs and Considerations

These users will interact with the web client application to monitor the ASV's status and performance during operations. The users may have varying levels of technical expertise, so the web client application will be designed to be user-friendly and accessible to a wide range of users which is also a Njord requirement. The system will also need to accommodate multiple users accessing the web client application simultaneously, especially during competitions where multiple team members and judges may need to access the data at the same time. The system is also used for diagnoistic purposes for both ISAAC Sim testing and real-life testing, so users may also include developers and testers who need to access the data for debugging and performance analysis. Various Stakeholders and sponsors may wish to view the graphical interface during testing or competitions, so the web client application will be designed to be visually appealing and informative for a wide range of audiences. Since we aren't collecting data or allowing control of the ASV through the web client, competing teams and judges can have the same level of access; thus, there will be no differentiation in user roles or permissions for non-technical users. All non-technical users will have access to the same data and features within the application, ensuring a consistent experience regardless of their affiliation with Humber ASV. Technical users such as the Humber ASV team must have access to create secure keys to distribute to other users of the web client application; allowing access. The focus of the user experience will be on providing clear and accessible information about the ASV's status and performance, rather than on differentiating access based on user roles or affiliations.

Operating Environment

The Base Station software will be designed to run on a Windows, Linux, or macOS computer that can connect to the ASV's access point. The web client application will be accessible through a web browser and will not require any additional software installation for user access. The system will need to be compatible with common web browsers such as Google Chrome, Mozilla Firefox, and Microsoft Edge to ensure accessibility for all users. The Base Station software will need to be able to run on a variety of hardware configurations, as the team may use different computers for development and testing. The system will also need to be designed to operate in environments with limited connectivity and potential interference, such as during maritime competitions where there may be multiple teams and devices operating in close proximity. Loon-E runs on a Jetson Orin with Ubuntu Jammy Jellyfish (22.04) and uses ROS Humble and Jetpack 5.0. A corresponding ROS node must be implemented to transmit data from the ASV to the Base Station. The Base Station and web client are isolated from the ROS enviroment and do not require compatibility considerations with it.

Functional Requirements

5.4.1 Web Client Application

- FR-1: The web client application shall display the ASV's speed over ground in meters per second.
- FR-2: The web client application shall display the ASV's heading.
- FR-3: The web client application shall display the ASV's current task status.
- FR-4: The web client application shall display the ASV's battery levels.
- FR-5: The web client application shall display the ASV's task data.
- FR-6: The web client application shall display a map showing the ASV's position and route.
- FR-7: The web client application shall display the signal strength of the connection between the ASV and the Base Station.

- FR-8: The web client application shall display a log from the ASV that includes relevant information about the ASV's operations and status.
- FR-9: The web client application shall display a power and rudder panel that shows the power allocation in the propeller and the rudder angle.
- FR-10: The web client application shall display received image recognition data from the ASV, such as object detection windows from the Zedx camera.
- FR-11: The web client application shall display image recognition data as boxes overlaid on the video feed from the Zedx camera.
- FR-12: The web client application shall be accessible through the latest (as of 2026) common web browsers and shall not require any additional software installation for user access.
- FR-13: The web client application shall have a border color that indicates the ASV's status, such as autonomous, remote control, standby, out of control, or lost connection.
- FR-14: The web client application shall use green border color to indicate autonomous mode.
- FR-15: The web client application shall use blue border color to indicate remote control mode.
- FR-16: The web client application shall use yellow border color to indicate standby mode.
- FR-17: The web client application shall use red border color to indicate out of control status.
- FR-18: The web client application shall use gray border color to indicate lost connection status.
- FR-19: The web client application shall update the border color in real-time based on the ASV's status.
- FR-20: The web client application shall have webpage with a text field that requires a valid token.
- FR-21: The web client application shall require a valid token to access and view the streaming data.
- FR-22: The web client application shall have an admin webpage that allows technical users to manage secure tokens for user access, including creating, revoking, and expiring tokens as needed to maintain security.

5.4.2 Base Station Application

- FR-23: The Base Station application shall connect to the ASV using Cyclone DDS communication protocols.
- FR-24: The Base Station application shall relay the received data through TCP from the ASV to the web client application for visualization and monitoring by users.
- FR-25: The Base Station application shall handle the authentication and security of the connection with salt-based hashing and token management to ensure that only authorized users can access the data.
- FR-26: The Base Station application shall host a website for the web client application.
- FR-27: The Base Station application endpoints shall reject unauthorized access attempts and display an appropriate error message to users who do not have valid tokens or permissions to access the application.

- FR-28: The Base Station shall use the endpoint */admin* for the admin webpage that allows technical users to create, revoke, or expire tokens as needed to maintain security.
- FR-29: The Base Station shall use the endpoint */admin/create_token* for the admin webpage to create secure tokens for user access.
- FR-30: The Base Station shall use the endpoint */admin/revoke_token* for the admin webpage to revoke secure tokens for user access.
- FR-31: The Base Station shall use the endpoint */admin/expire_token* for the admin webpage to expire secure tokens for user access.
- FR-32: The Base Station shall use the endpoint */admin/tokens* for the admin webpage to get a list of all active tokens for user access.
- FR-33: The Base Station shall use the endpoint */client* for the web client application to access the streaming data with a valid token.
- FR-34: The Base Station shall use the endpoint */uh_oh* to handle unauthorized access attempts and display an appropriate error message to users who do not have valid tokens or permissions to access the application.
- FR-35: The Base Station shall allow technical users to create a admin user account through terminal commands.
- FR-36: The Base Station application shall allow the admin user to create, revoke, or expire tokens as needed to maintain security.

5.4.3 ASV

- FR-37: The ASV shall collect data from various sensors and systems on the ASV, such as telemetry data, sensor readings, and video feeds from the Zedx camera.
- FR-38: The ASV shall transmit the collected data to the Base Station application using Cyclone DDS communication protocols.

6 Software Development

This section provides information on how to set up the development environment, run the software, and contribute to the Loon-E ASV project. It also includes relevant documentation and resources for developers to effectively contribute to the project.
In order to contribute to the project, developers will need to set up their development environment with the necessary software and tools. The required software and tools for development are listed in Figure 7

- GitHub
- Tailscale
- Rust Desk
- Git
- ROS2 Humble
- Stereolab's ZedSDK 5.3
- Putty (optional, and for Windows only)
- FileZilla (optional, and for Windows only)
- Python 3.10
- Discord (Used for communications)
- LiveTeX (distribution of LaTex)
- NPM (may change later)
- Microsoft Teams (Primary Document Sharing)
- Docker
- Any preferred LaTex editor
- Any Preferred IDE
- Any Standard Preferred Web Browser

Figure 7: Software and tools required for development.

Development Resources

A few primary resources are available for you to get started with development:

- documentation
- codebase access
- communication channels

At bare minimum, you'll require Latex for in-depth documentation (I.E this documentation), an IDE for code development, and a web browser for accessing the codebase and communication channels. Further details on what tools you need are in Figure 7.

6.1.1 Documentation

Documentation is split into various channels, each serving a different purpose.

This Document This public document serves as the primary source of information for the Loon-E ASV project, providing comprehensive details on the design, operation, and maintenance of the ASV. It is intended to be a valuable resource for stakeholders and developers involved in the project. It is designed to be modular and easily navigable, allowing readers to quickly find the information they need and so multiple developers can work on different sections without conflicts.

GitHub Wiki The public GitHub Wiki serves as a collaborative platform for developers to share information, best practices, and updates related to the project. It is a living document that evolves with the project and provides a space for developers to contribute their knowledge and insights.

GitHub README The public GitHub README file provides an overview of the project, including its purpose, features, and instructions for getting started. It serves as an introduction to the project for new developers and provides essential information for understanding the project's goals and structure.

Github Issues and Pull Requests GitHub Issues and Pull Requests serve as a platform for developers to report bugs, suggest features, and contribute code to the project. They provide a structured way for developers to collaborate and track the progress of the project.

GitHub Project Board The GitHub Project Board is used only for Software related project management and task tracking. It provides a visual representation of the project's progress, allowing developers to see what tasks are in progress, what tasks are completed, and what tasks are upcoming. Debate exists on whether this or Microsoft Teams Kanban board should be the primary project management tool, but for now, the GitHub Project Board is used for software-related tasks, while the Microsoft Teams Kanban board is used for everything else.

Teams Onedrive The Teams Onedrive serves as a centralized repository for project-related documents, including design files, meeting notes, and other resources. It provides a secure and organized space for storing and sharing project materials among team members. You'll need to be added to the Teams Onedrive by a project manager to access it, so reach out to one if you need access (see discord).

6.1.2 Codebase

GitHub Repository The GitHub repository serves as the primary codebase for the Loon-E ASV project, hosting all source code, and some related resources; however, it is reccomended to use this document for a single source of truth. It provides a collaborative platform for developers to contribute code, track changes, and manage the project's development.

Branching Strategy No current branching strategy exists, but we are in the process of implementing one. The proposed branching strategy includes a main branch for stable releases, a development branch for ongoing work This strategy helps to organize the codebase and facilitate collaboration among developers.

Code Reviews Code reviews are conducted through GitHub Pull Requests, allowing team members to review and provide feedback on proposed changes before they are merged into the main codebase. This process helps to maintain code quality and ensure that changes align with the project's goals and standards.

Pull Request Process The pull request process involves creating a new branch for the proposed changes, making the necessary code modifications, and then submitting a pull request for review. Team members can comment on the pull request, suggest improvements, and ultimately approve or request changes before the code is merged into the main branch. It is important to follow the established pull request process to ensure that contributions are properly reviewed and integrated into the project.

6.1.3 Communication Channels

Discord The private Discord server serves as the primary communication channel for the Loon-E ASV project, allowing team members to collaborate, share updates, and discuss project-related topics in real time. It provides a platform for developers to ask questions, share ideas, and stay connected with the rest of the team. You'll need to be added to the Discord server by a project manager to access it, so reach out to our email.

Email Email is used for formal communications and is not for development discussions. It is used mostly for communicating with external stakeholders; however, if you wish to join us then you can reach out to our email and we will get back to you as soon as possible. Our email is: mechatronicsclub@humber.ca

Microsoft Teams Microsoft Teams is used for document sharing and collaboration among team members. It provides a centralized space for storing and sharing project-related documents, including design files, meeting notes, and other resources. It also tracks the team Kanban board, which is used for project management and task tracking. You'll need to be added to the Microsoft Teams by a project manager to access it, so reach out to one if you need access (see discord).

Contribution Guidelines

Refer to each github repository for specific contribution guidelines, but in general, contributions to the project should follow these guidelines:

- Be respectful and collaborative in all interactions with other team members.
- Follow the established coding standards and best practices for the project.
- Use the pull request process for all code contributions, and ensure that your code is properly reviewed before it is merged into the main codebase.
- Report any issues or bugs you encounter in the GitHub Issues, and provide as much detail as possible to help others understand and address the issue.
- Stay engaged with the project and communicate regularly with other team members to ensure that your contributions align with the project's goals and priorities.

6.2.1 Code of Conduct

To maintain a positive and inclusive environment for all contributors, we have established a code of conduct that all team members are expected to follow; See Figure 8 for details.

- Respect and Kindness: Treat others with respect and kindness. Disagreements are fine, but personal attacks, harassment, or any form of disrespectful behavior will not be tolerated.
- English Only: To maintain a cohesive and inclusive community, please keep all communications in English.
- No NSFW/Extreme Content: This is a robotics server focused on learning and collaboration. Any content or discussions deemed NSFW (Not Safe For Work) or extreme in nature are strictly prohibited.
- Unsolicited DMs: Respect the privacy of others. Do not send unsolicited direct messages to members without their consent. If you wish to reach out to someone, please do so in a respectful manner and only after obtaining permission in the server channels.

Figure 8: Code of Conduct

7 Docker Use

Docker is a powerful tool for creating, deploying, and running applications in containers. It allows you to package your application with all of its dependencies into a standardized unit for software development. This can be particularly useful for ROS 2 development, as it can help ensure that your development environment is consistent across different machines and operating systems.

> **WARNING**
>
> It is prerequisite to have an understanding of Linux systems before using Docker.

> **NOTE**
>
> This document assumes you have already set up your environment variables as described in the Environment Variables documentation and that you have installed Docker. If you have not done so, please refer to that documentation before proceeding.

Images

Docker images are the basis of Docker containers. They are read-only templates that contain the instructions for creating a Docker container. You can create your own Docker images or use pre-built images from Docker Hub. For ROS 2 development, you can use the official ROS 2 images provided by Open Robotics or, for NVIDIA Jetson devices, images provided by NVIDIA. These images come pre-configured with ROS 2 and the necessary dependencies, making it easier to get started with development.
We have used our own custom image based on the official ROS 2 image, which includes additional tools and libraries that we find useful for our development process. You can find our custom Dockerfile when we release our Loon-Env repository.

> **NOTE**
>
> For our NVIDIA Orin Nano, we are using `nvcr.io/nvidia/isaac/ros:aarch64-ros2_humble_4c0c55dddd2bbcc3e8d5f9753bee634c`, which is based on the official ROS-2 Humble image and includes additional tools and libraries for Loon-E's development on NVIDIA Jetson devices.

Volumes

Docker volumes are a way to persist data generated by and used by Docker containers. They allow you to store data outside of the container's filesystem, which can be useful for sharing data between the host machine and the container or for persisting data across container restarts. When using Docker for ROS 2 development, you can use volumes to share your ROS 2 workspace between the host machine and the container. This allows you to edit your code on the host machine and have those changes reflected in the container without needing to rebuild the image.
Because Loon-E uses Zedx we have to make volumes to ensure the Zedx SDK and its dependencies are properly shared between the host machine and the container. This is necessary because the Zedx SDK

requires access to certain hardware resources, such as the GPU, which may not be available within the container. By using volumes, we can ensure that the necessary files and libraries are accessible to the container, allowing us to use the Zedx SDK for vision processing in our ROS 2 development.
Following the example on Stereolabs, we need to set the following volumes.

7.2.1 Hardware & Device Volumes

Volume Name	Mount Path	Purpose
/dev/	/dev/	This is the device file for the Zedx camera. It allows the container to access the camera hardware.
/tmp/	/tmp/	Temporary directory that the Zedx SDK uses for storing temporary files. It allows the container to access these files as needed.

Table 2: Docker Hardware Volumes for Zedx SDK

7.2.2 SDK Configuration Volumes

Volume Name	Mount Path	Purpose
/var/nvidia/nvcam/settings/	/var/nvidia/nvcam/settings/	Directory where the Zedx SDK stores its configuration files. It allows the container to access these files as needed.
/etc/systemd/system/zed_x_daemon.service	/etc/systemd/system/zed_x_daemon.service	Systemd service file for the Zedx daemon. It allows the container to manage the Zedx daemon as needed.
$HOME/zed_docker_ai/	/usr/local/zed_docker_ai/	Host directory used for storing any additional files or data related to ROS 2 development with the Zedx SDK. It allows easy sharing between host and container.

Table 3: Docker SDK Configuration Volumes for Zedx SDK

7.2.3 Display & X11 Volumes

Volume Name	Mount Path	Purpose
$HOME/.Xauthority	/root/.Xauthority	X authority file for the host machine. It allows the container to access the X server for running graphical applications.

Table 4: Docker Display Volumes for Zedx SDK

Options

When running a Docker container for ROS 2 development, there are several options that you may need to set to ensure that the container has the necessary permissions and access to resources. These options can include runtime settings, network settings, and GPU access.

Option Name	Purpose
–runtime nvidia	Allows the container to use the NVIDIA runtime, necessary for accessing GPU resources required by the Zedx SDK.
-it	Interactive terminal for running commands and debugging within the container.
–privileged	Gives the container additional privileges, which may be necessary for accessing hardware resources or managing system services.
–net=host	Uses the host machine's network stack, useful for communication between the container and other devices on the network.
–ipc=host	Uses the host machine's IPC namespace, necessary for applications that require shared memory or other IPC mechanisms.
–gpus all	Allows the container to access all available GPU resources on the host machine.
–pid=host	Uses the host machine's PID namespace, useful for applications that require access to the host's process information or for debugging.

Table 5: Docker Options for ROS 2 Development

Environment Variables

When running a Docker container for ROS 2 development, you may need to set certain environment variables to ensure that the container has the necessary configuration and access to resources. These environment variables can include display settings for graphical applications, ROS domain ID for communication, and RMW implementation for ROS 2 middleware.

Environment Variable	Purpose
DISPLAY	Allows the container to access the host machine's display, useful for running graphical applications or visualizing data from the Zedx camera.
XAUTHORITY	Allows the container to access the host machine's X authority file, necessary for running graphical applications that require access to the X server.
`ROS_DOMAIN_ID`	Sets the ROS domain ID for the container, necessary for ensuring that the container can communicate with other ROS nodes on the same network.
`ROS_LOCALHOST_ONLY`	Limits ROS communication to localhost only; useful to prevent interference with other ROS systems on the network.
`RMW_IMPLEMENTATION`	Specifies which ROS middleware implementation to use within the container, ensuring compatibility with ROS nodes and libraries in use.

Table 6: Environment Variables for Docker ROS 2 Development

Example

Here is an example of how to run a Docker container for ROS 2 development with the necessary volumes, options, and environment variables:

```
docker run --runtime nvidia -it --privileged --net=host --ipc=host --gpus all --pid=host \
 -v /dev/:/dev/ \
 -v /tmp/:/tmp/ \
 -v /var/nvidia/nvcam/settings/:/var/nvidia/nvcam/settings/ \
 -v ${HOME}/zed_docker_ai/:/usr/local/zed_docker_ai/ \
 -v ${HOME}/.Xauthority:/root/.Xauthority \
 -e DISPLAY=$DISPLAY \
 -e XAUTHORITY=$XAUTHORITY \
 -e ROS_DOMAIN_ID=$ROS_DOMAIN_ID \
 -e ROS_LOCALHOST_ONLY=$ROS_LOCALHOST_ONLY \
 -e RMW_IMPLEMENTATION=$RMW_IMPLEMENTATION \
 --name loon-e-dev-container \
 loon-e-dev-image:latest
```

> **NOTE**
>
> `ROS_DOMAIN_ID`, `ROS_LOCALHOST_ONLY`, and `RMW_IMPLEMENTATION` should be set in your host environment before running the container; they will be passed into the container as environment variables. This ensures that the ROS 2 configuration within the container matches that of the host environment.

8 Colcon Setup

Background

`colcon` is an iteration on the ROS build tools `catkin_make`, `catkin_make_isolated`, `catkin_tools` and `ament_tools`. For more information on the design of `colcon` see the design document.
The source code can be found in the colcon GitHub organization.

8.1.1 Requirements

Ensure you already installed ROS 2. If you have not installed ROS 2 follow the installation instructions.

Installation Linux

You can install `colcon` through the `apt` package manager with the command:

```
sudo apt install python3-colcon-common-extensions
```

Installation macOS

You can install `colcon` with Python's `pip`:

```
python3 -m pip install colcon-common-extensions
```

Installation Windows

You can install `colcon` with Python's `pip` on Windows:

```
pip install -U colcon-common-extensions
```

9 Environment Variables Setup

For all operating systems, you will need to set up some environment variables to ensure ROS 2 operates correctly. This includes sourcing the ROS 2 setup files and setting any necessary ROS 2 environment variables.
First, source the setup files for your OS, then set the environment variables described below.

Domain ID

See the domain ID article for details on ROS domain IDs.

Localhost Only

By default, ROS 2 communication is not limited to localhost. The environment variable `ROS_LOCALHOST_ONLY` allows you to limit ROS 2 communication to localhost only. This means your ROS 2 system, and its topics, services, and actions will not be visible to other computers on the local network.

RMW Implementation

ROS 2 supports multiple RMW implementations, which are the underlying middleware that handles communication. The default RMW implementation is `rmw_fastrtps_cpp`. Setting `RMW_IMPLEMENTATION` allows you to choose which implementation ROS 2 should use.

Example

```
ROS_DOMAIN_ID=0 # This is also the default
ROS_LOCALHOST_ONLY=1 # This is also the default
RMW_IMPLEMENTATION=rmw_fastrtps_cpp # This is also the default
```

> **WARNING**
>
> The exact commands depend on your operating system and shell. If you're having problems, ensure the file path leads to your installation and that you're using the correct syntax for your shell.

> **INFO**
>
> These variables must match if run in a container. If you set `ROS_LOCALHOST_ONLY=1` in your host environment, you must also set it in your container environment. If you set `ROS_DOMAIN_ID=0` in your host environment, you must also set it in your container environment. There is additional information on container use in the Docker documentation.

Linux

Begin by sourcing the ROS 2 setup file and setting the necessary environment variables:

```
source /opt/ros/humble/setup.bash
export ROS_DOMAIN_ID=0
export ROS_LOCALHOST_ONLY=1
export RMW_IMPLEMENTATION=rmw_fastrtps_cpp
```

> **NOTE**
>
> To set environment variables persistently, add them to your shell startup script:

```
echo "export ROS_DOMAIN_ID=0 ROS_LOCALHOST_ONLY=1 RMW_IMPLEMENTATION=rmw_fastrtps_cpp" >> ~/.bashrc
```

Windows

To source the ROS 2 local setup on Windows PowerShell:

```
call C:\dev\ros2\local_setup.bat
```

If you don't want to source the setup file every time you open a new shell, create a folder in *My Documents* called *WindowsPowerShell* and create a file named `Microsoft.PowerShell_profile.ps1` containing the path to your setup script. You may need to run 'Unblock-File' on the script to avoid repeated permission prompts.

> **WARNING**
>
> The exact command depends on where you installed ROS 2. If you're having problems, ensure the file path leads to your installation.

Check Environment Variables

Sourcing ROS 2 setup files will set several environment variables necessary for operating ROS 2. If you ever have problems finding or using your ROS 2 packages, make sure that your environment is properly set up using the following commands.

macOS and Linux

```
printenv | grep -i ROS
```

Windows

```
set | findstr -i ROS
```

Check that variables like `ROS_DISTRO` and `ROS_VERSION` are set. Example values:

```
ROS_VERSION=2
ROS_PYTHON_VERSION=3
ROS_DISTRO=humble
```

If the environment variables are not set correctly, return to the ROS 2 package installation section of the installation guide you followed. If you need more specific help, you can get answers from the community.

Troubleshooting

Check the DDS settings if you are having trouble with ROS 2 communication. You may need to set additional environment variables depending on your DDS implementation and network configuration.

10 Remote Connections

Local development is done on the ASV's primary computer, which is a Jetson Orin. Remote access to the primary computer is necessary for monitoring the ASV's operations and for development purposes.
In order to connect, the lovely folks at TailScale [4] have set up a secure VPN connection to the ASV's primary computer, allowing developers to access the system remotely from their own devices. This VPN connection ensures that developers can securely access the ASV's primary computer from anywhere, enabling them to monitor the ASV's operations, access logs, and perform development tasks without needing to be physically present at the ASV's location. At cost, local development on a small device is not scalable for larger teams, fortunately, our software team is relatively small and the VPN connection provides a convenient solution for remote access to the ASV's primary computer.

TailScale Connection

To connect to the ASV's primary computer using TailScale, developers need to follow these steps:

1. Install the TailScale client on their local device (available for Windows, macOS, Linux, iOS, and Android).
2. Create a Github account (if they don't have one already)
3. Notify the software team lead of their Github username so that they can be added to our Github organization.
4. Sign in to TailScale using their Github credentials.[5]
5. Once signed in, they will see the ASV's primary computer listed as a device in their TailScale dashboard.
6. Click on the ASV's primary computer (named "ubuntu") to establish a secure VPN connection.
7. Once connected, developers can access the ASV's primary computer as if they were on the same local network, allowing them to monitor and develop the software remotely.

RustDesk Connection

In addition to the TailScale VPN connection, developers can also use RustDesk for remote desktop access to the ASV's primary computer. RustDesk is an open-source remote desktop software that allows users to access and control a computer remotely. To use RustDesk for remote desktop access, developers need to follow these steps:

1. Install the RustDesk client on their local device (available for Windows, macOS, Linux, iOS, and Android).
2. Obtain the RustDesk ID and password for the ASV's primary computer from the software team lead.
3. Open the RustDesk client and enter the ASV's primary computer's RustDesk ID and password to establish a remote desktop connection.

[4] TailScale is a popular VPN solution that provides secure and easy-to-use remote access to devices.
[5] TailScale let us use their service so long as we are open source and connected to the Github organization. Developers will need to sign in with their Github account to access the VPN.

11 Running the Software

Refer to Section 10 for instructions on how to remotely connect to the ASV's primary computer; this is needed for running software on the ASV.
Once connected to the primary computer, a custom script has been created to simplify the process of running the software. To start the software, developers can run the following command in the terminal:

```
LoonE --start
```

This command will build and execute a docker container from configurations installed from the custom script's install features.

> **INFO**
>
> The LoonE command is installed with the */install.bash* located in the repository, which is run as part of the setup process. This command is a custom script that simplifies the process of starting the software by automating the necessary steps to build and run the software in a Docker container.

> **WARNING**
>
> The LoonE command may also freeze the terminal and cause a temporary 10 minute loss of connection to the ASV, but this is normal as the software is starting up and may take some time to build. It is not recommended to stop the process of building the container.

[6]

> **WARNING**
>
> Changes in container will not be saved

You may need to enter the container in multiple terminals to run different processes. Follow the instructions below to enter the container:

```
docker ps # to get the container id or name
docker exec -it <container_id_or_name> /bin/bash
```

For our environment it is set to build ZedXWrapper from our main repository instead of stereolabs' repository. This is because Stereolabs' repository is actively maintained and may introduce breaking changes that can disrupt our development process. By building from our repository, we have more control over the software and can ensure that it remains stable and compatible with our system, allowing for a smoother development experience. Additionally, it makes it easier for us to set the configuration for the ZedX camera, which is crucial for our ASV's operations.
To run implemented features within the container, the current command is as follows:

```
ros2 launch loon_e_coms coms_launch.py
```

[6] A docker container either downloads the scripts or establishes a volume from the host which has the scripts.

This command will launch the ROS2 nodes—including Zedx-Wrapper—for the Loon-E ASV's communication system, allowing the ASV to send and receive data as part of its operations. See the launch file for more details on what processes are launched and how they are configured.

> **INFO**
>
> You can learn more about launch files in ROS here.

Troubleshooting

> **NOTE**
>
> For most issues, you may benefit from reading Section 7 or 8 to ensure your environment is set up correctly.

If you encounter issues while running the software, here are some common troubleshooting steps:

11.1.1 Camera not connecting

If the ZedX camera is not connecting or functioning properly, try the following steps:

- Ensure that the ZedX camera is properly connected to the primary computer and powered on.
- Check the camera's physical connection and power status, and ensure that it is securely plugged in.
- Verify that the necessary drivers and software for the ZedX camera are installed and up to date

```
apt list --installed | grep zed
```

  This should show the zedlink-duo and ros-humble-zed-msgs packages

- Check diagnostics

```
ZED_Diagnostic
```

  This should show diagnostic messages from the ZedX camera, which can help identify any issues with the camera's operation.

> **WARNING**
>
> If in a container the diagnostic tool will not detect the driver, you will need to run it on the host machine

> **NOTE**
>
> If the camera is not connecting, it may be due to a hardware issue, driver issue, or configuration issue. Checking the physical connection, verifying software installation, and reviewing diagnostic messages can help identify and resolve the problem.

11.1.2 ROS2 nodes not communicating

If the ROS2 nodes are not communicating properly, try the following steps:

- Ensure that all ROS2 nodes are properly launched and running.
- Check the ROS2 topic list to verify that the expected topics are being published and subscribed to:

```
ros2 topic list
```

- Use ROS2 topic echo to check the data being published on specific topics:

```
ros2 topic echo <topic_name>
```

- Check for any error messages in the terminal where the nodes are running, which may indicate issues with node communication or configuration.
- Verify that the ROS2 middleware (Cyclone DDS) is properly configured and running, as it is responsible for facilitating communication between nodes.
- Ensure the topics are being run on the same Domain ID, as ROS2 uses DDS which requires all nodes to be on the same Domain ID to communicate.
- Check for any firewall or network issues that may be preventing communication between nodes, especially if running across multiple machines.
- Verify that the ROS2 environment variables are properly set, as incorrect environment configurations can lead to communication issues between nodes.
- If using Docker, ensure that the container has the necessary network configurations to allow communication between nodes, especially if nodes are running in different containers or on the host machine.

11.1.3 ROS2 topic list waiting indefinitely

Your ROS2 topic list may be waiting indefinitely due to a misconfiguration in the ROS2 environment variables, network issues, or problems with the ROS2 middleware (Cyclone DDS). Here are some steps to troubleshoot this issue:

- Check that the ROS2 environment variables are properly set, especially `ROS_DOMAIN_ID`, `ROS_LOCALHOST_ONLY`, and `RMW_IMPLEMENTATION`. Incorrect settings can lead to communication issues and cause the topic list to wait indefinitely. You can read about them in Section 8.

- Verify that the ROS2 middleware (Cyclone DDS) is properly configured and running, as it is responsible for facilitating communication between nodes. Issues with the middleware can lead to communication problems and cause the topic list to hang.
- Check for any firewall or network issues that may be preventing communication between nodes, especially if running across multiple machines. Network misconfigurations can lead to communication failures and cause the topic list to wait indefinitely.
- Ensure that all ROS2 nodes are properly launched and running, as missing or non-functional nodes can lead to communication issues and cause the topic list to hang.
- If using Docker, ensure that the container has the necessary network configurations (see Section 7) to allow communication between nodes, especially if nodes are running in different containers or on the host machine. Docker network misconfigurations can lead to communication failures and cause the topic list to wait indefinitely.

11.1.4 RVIZ not displaying data

> **NOTE**
>
> RVIZ has been modified for stereocameras under Zed ROS 2 Wrapper, use the modified version of RVIZ for best results.

RVIZ is a visualization tool for ROS2 that allows users to visualize sensor data, robot models, and other information. If RVIZ is not displaying data properly, try the following steps:

- Ensure that RVIZ is properly installed and launched.
- Check that the correct ROS2 topics are being visualized in RVIZ, and that they are being published with the expected data.
- Verify that the RVIZ configuration is set up correctly, including the fixed frame and any necessary transforms.
- Check for any error messages in the terminal where RVIZ is running, which may indicate issues with data visualization or configuration.
- Ensure that the necessary RVIZ plugins are installed and configured to visualize the specific types of data being published by the nodes.

Other issues may arise such as:

- Nodes not launching properly
- Data not being published or subscribed to as expected
- Network communication issues between nodes
- Hardware issues with sensors or the primary computer
- Zedx Configuration issues (likely cause)

11.1.5 Simulation issues

Currently there are only one setup for our branch of the ROS2 wrapper for the ZedX camera, which is to build the wrapper from our repository and not to use in ROS2 Isaac SIM bridge. This is because we haven't done that yet.

12 ROS2 Design

We are currently undergoing changes to our ROS2 Architecture to better fit our needs and constraints. The current architecture consists of 7 nodes: Control, Map, Movement, Planning, Base Station, ROS2 Publish Clock, and Zedx Camera. The data distribution between these nodes is illustrated in Table 8. The proposed architecture consists of 8 nodes: Control, Map, Movement, Base Station, ROS2 Publish Clock, Bound_Box_Video, and Zedx Camera. The data distribution between these nodes is illustrated in Table 9.

Node Name	Purpose
Control Node	Executes control logic and navigation commands
Map Node	Maintains and updates local environmental map
Movement Node	Manages motor control and vehicle movement
Planning Node	Computes navigation paths to destination
Base Station Node	Monitors and logs all system data
ROS2 Publish Clock Node	Synchronizes simulation timing across nodes
Zedx Camera Node	Processes stereo vision and object detection

Table 7: Primary Computer ROS2 Nodes

Node Name	Subscriptions	Publications
Control	map (local), objects, locations	PWM
Map	obstacles, locations	map (local)
Movement Node	PWM	not implemented
Base Station	ALL	None
Planning node	map (local), destination	path
ROS2 Publish Clock	None	clock
Zedx Camera	clock[7]	objects, pose, image_rect_color, map (global)

Table 8: Current Primary Computer ROS2 Nodes Data Distribution

> **NOTE**
>
> Movement needs pose and odometry data to move. The Zedx camera needs to publish objects and mapping receive it.

Node Name	Subscriptions	Publications
Control	map (local), map (global), objects	twist, status
Map	pose, odom, objects[8]	map (local)
Movement Node	twist, status	health (movement), power
Base Station	ALL	None
ROS2 Publish Clock	None	clock
Zedx Camera	clock	objects, pose, image_rect_color, map (global)
Bound_Box_Video	objects, image_rect_color	video_feed

Table 9: Proposed Computer ROS2 Node Data Distribution

[7] The Zedx Camera node subscribes to the clock topic to synchronize its data publishing with Isaac SIM.[10]
[8] Zedx Objects contains location and classification data [8].

A Glossary

Autonomous Surface Vehicle (ASV) An autonomous surface vehicle (ASV) is an unmanned vessel that operates on the sea surface without real-time input or control from human operators.[12]

Base Station A fixed radio communications station that can communicate with mobile radio devices in its neighbourhood. Forms of radio communication that have base stations include radio wireless data and mobile telephony...[1]

Loon-E Humber ASV's current ASV.[2]

HumberASV Humber Polytechnic's Mechatronics club. A multidisciplinary team of passionate Humber Polytechnic students and professionals dedicated to advancing autonomous maritime technology.[2]

LIDAR LiDAR, an acronym for "light detection and ranging," is a remote-sensing technology that uses laser beams to measure precise distances and movement in an environment, in real time.[3]

Robot Operating System (ROS) The Robot Operating System (ROS) is a set of software libraries and tools that help you build robot applications.[5]

B Bibliography

[1] Andrew Butterfield and John Szymanski. *base station*. 2018. DOI: 10.1093/acref/9780198725725.013.0348. URL: https://www.oxfordreference.com/view/10.1093/acref/9780198725725.001.0001/acref-9780198725725-e-348.

[2] HumberASV. *Our Team*. 2026. URL: https://humberasv.ca/team (visited on 03/17/2026).

[3] IBM. *What is LIDAR?* 2026. URL: https://www.ibm.com/think/topics/lidar (visited on 03/17/2026).

[4] Open Robotics. *DDS implementations*. 2026. URL: https://docs.ros.org/en/humble/Installation/RMW-Implementations/DDS-Implementations.html (visited on 04/16/2026).

[5] Open Robotics. *ROS*. 2026. URL: https://www.ros.org/ (visited on 03/17/2026).

[6] Open Robotics. *ROS2 Documentation: Humble*. 2026. URL: https://docs.ros.org/en/humble/index.html (visited on 04/16/2026).

[7] Amelia Soon et al. *RoboBoat 2026 Technical Design Report – Humber ASV Loon-E*. Toronto, ON, Canada, 2026. URL: https://humberasv.ca/docs (visited on 04/16/2026).

[8] STEREOLABS. *Custom Object Detection and Tracking with ROS 2*. 2026. URL: https://www.stereolabs.com/docs/ros2/custom-object-detection (visited on 04/16/2026).

[9] STEREOLABS. *DDS Middleware and Network tuning*. 2026. URL: https://www.stereolabs.com/docs/ros2/dds-and-network-tuning/ (visited on 04/16/2026).

[10] STEREOLABS. *Using ZED with ROS 2 and Isaac Sim*. 2026. URL: https://www.stereolabs.com/docs/isaac-sim/ros2_integration (visited on 04/16/2026).

[11] STEREOLABS. *Zed SDK Documentation*. 2026. URL: https://www.stereolabs.com/docs/ (visited on 04/16/2026).

[12] Arthur Trembanis, Mark Lundine, and Kaitlyn McPherran. "Coastal Mapping and Monitoring". In: *Encyclopedia of Geology (Second Edition)*. Ed. by David Alderton and Scott A. Elias. Second Edition. Oxford: Academic Press, 2021, pp. 251–266. ISBN: 978-0-08-102909-1. DOI: https://doi.org/10.1016/B978-0-12-409548-9.12466-2. URL: https://www.sciencedirect.com/science/article/pii/B9780124095489124662.